HASHDIT

HashDit

Smart Contract Security

Venus PrimeV2

Security Audit Report

Venus

12 May 2026 – 22 May 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	12 May 2026 — 22 May 2026
Overall Risk [Optional]	Medium
Project Name	Venus
Github	https://github.com/VenusProtocol/venus-protocol/pull/670
Commit	feat/prime-v2

Smart Contract List

No.	Contract Name	Verdict	Details
1	PrimeV2.sol	Medium	[M01] [L01] [L04] [I01] [I02] [I03] [I06] [I07] [I08] [I09] [I11] [I12] [I13] [I14]
2	PrimeV2Storage.sol	Green	-
3	PrimeLeaderboard.sol	Medium	[M02] [L03] [I04] [I05] [I10]
4	PrimeLeaderboardStorage.sol	Low	[L02]

Findings

[M01] Inconsistent handling of boundary alpha values in Cobb-Douglas score calculation	
Contract	PrimeV2.sol
Severity	Medium
Description	**Description** The Prime V2 implementation relaxes the bounds on the alpha parameter compared to V1. In V1, _checkAlphaArguments reverts when _alphaNumerator >= _alphaDenominator or _alphaNumerator == 0, restricting alpha to the strict open interval 0 < α < 1. JavaScript `/** * @notice Verify new alpha arguments * @param _alphaNumerator numerator of alpha. If alpha is 0.5 then numerator is 1 * @param _alphaDenominator denominator of alpha. If alpha is 0.5 then denominator is 2 * @custom:error Throw InvalidAlphaArguments if alpha is invalid */ function _checkAlphaArguments(uint128 _alphaNumerator, uint128 _alphaDenominator) internal pure { if (_alphaNumerator >= _alphaDenominator || _alphaNumerator == 0) { revert InvalidAlphaArguments(); } }` In V2, updateAlpha only reverts when alphaDenominator == 0 or alphaNumerator > alphaDenominator, which expands the accepted range to the closed interval 0 <= α <= 1, including the endpoints α=0 and α=1. Per the Prime documentation, alpha is the weight assigned to XVS staking versus capital (supply and borrow) in the Cobb Douglas score formula, and the protocol may legitimately want to set alpha to the endpoints to fully incentivize one side. When α=1, capital has zero weight, since capital^0 = 1, so a user's score should depend solely on their XVS stake (although unlikely), and the user should not need to supply or borrow in a market to receive a nonzero score.

	However, the Cobb Douglas implementation in Scores contains an early return that yields 0 whenever xvs == 0 || capital == 0. This short circuit conflicts with the mathematical meaning of the boundary alpha values now permitted by the validation. With α=1, any user who has staked XVS but holds no capital in a market will incorrectly receive a score of 0 instead of a score driven entirely by their stake. Symmetrically, with α=0, a user with capital but no XVS will receive 0 instead of a score driven entirely by their capital. The validation and the score formula are therefore inconsistent, and users will be denied rewards they are entitled to under the configured alpha. JavaScript `/** * @notice Validate alpha arguments * @param alphaNumerator_ Alpha numerator * @param alphaDenominator_ Alpha denominator */ function _checkAlphaArguments(uint128 alphaNumerator_, uint128 alphaDenominator_) internal pure { if (alphaDenominator_ == 0 || alphaNumerator_ > alphaDenominator_) { revert InvalidAlphaArguments(); } }`
Recommendation	Align the score function with the validation. Either restore the strict V1 bounds in updateAlpha so that α=0 and α=1 are rejected, keeping the existing zero short circuit consistent, or remove the unconditional if (xvs == 0 || capital == 0) return 0;
Status	Fixed in commit 5fcc3f8, the check is reverted to the original _checkAlphaArguments, similar to as in Prime (V1), preventing possible configurations mentioned in the issue.

[M02]_compactDeposits Pass 1 overestimates effective stake when longer multiplier tiers are added after compaction	
Contract	PrimeLeaderboard.sol
Severity	Medium
Description	**Description** The PrimeLeaderboard._compactDeposits function performs a two pass compaction of a user's deposit stack. 1. Pass 1 walks the user's deposits and merges every deposit whose holdingDuration >= maxTierDuration into a single deposit at index 0. The merged entry uses the sum of the amounts and the earliest of the merged

timestamps, with a code comment stating that the earliest timestamp is preserved so the "true hold duration is retained" if tiers are later increased.

JavaScript

```
     // If we merged any deposits, add a single merged deposit at the
beginning.
        // The O(n) shift preserves oldest-first ordering required for
LIFO withdrawals:
        // _recordWithdrawal pops from the end (newest first), so the
merged max-tier
        // deposit must stay at index 0 to be withdrawn last.
        if (mergedAmount > 0) {
            for (uint256 i = writeIndex; i > 0; ) {
                unchecked {
                    --i;
                }
                deposits[i + 1] = deposits[i];
            }

            // Insert merged deposit at index 0, preserving the earliest
original timestamp
            // so that if multiplier tiers are later increased, the true
hold duration is retained
            deposits[0] = Deposit({ amount: mergedAmount.toUint128(),
timestamp: earliestTimestamp, _reserved: 0 });

            writeIndex++;
        }
```

2. Pass 2 (_compactByTier), used as a fallback when the user is still at the deposit cap, instead buckets deposits by tier and writes back an amount weighted average timestamp per bucket.

JavaScript

```
  function _compactByTier(Deposit[] storage deposits) internal {
        uint256 tiersCount = _multiplierDurations.length + 1; // base
tier + configured tiers
        uint256[] memory tierAmounts = new uint256[](tiersCount);
        uint256[] memory tierWeightedTimestamps = new
uint256[](tiersCount);

        // Group deposits by tier using weighted timestamp sum
        uint256 depositsLength = deposits.length;
        for (uint256 i; i < depositsLength; ) {
            Deposit storage d = deposits[i];
```

```
            uint256 holdingDuration = block.timestamp -
uint256(d.timestamp);
            uint256 tierIndex = _getTierIndex(holdingDuration);

            tierAmounts[tierIndex] += uint256(d.amount);
            tierWeightedTimestamps[tierIndex] += uint256(d.amount) *
uint256(d.timestamp);

            unchecked {
                ++i;
            }
        }

        // Write back non-empty tiers, oldest first (highest tier index
= longest duration)
        uint256 newCount = 0;
        for (uint256 i = tiersCount; i > 0; ) {
            unchecked {
                --i;
            }
            if (tierAmounts[i] > 0) {
                uint64 avgTimestamp = uint64(tierWeightedTimestamps[i] /
tierAmounts[i]);
                deposits[newCount] = Deposit({
                    amount: tierAmounts[i].toUint128(),
                    timestamp: avgTimestamp,
                    _reserved: 0
                });
                unchecked {
                    ++newCount;
                }
            }
        }

        while (deposits.length > newCount) {
            deposits.pop();
        }
    }
```

In Pass 1, the earliest timestamp choice in Pass 1 is only lossless for the single oldest deposit. For every other deposit merged into the single entry, the original deposit time is discarded. If governance later extends the tier configuration by adding a new tier whose duration is greater than the maxTierDuration at the time of compaction, the merged deposit appears to have been held since the earliest timestamp, and the entire merged amount is credited at the new top tier multiplier even though some of the underlying XVS was deposited far later and individually would not yet qualify for that new tier.

Consider the following example where a user stakes 100 XVS into the XVS vault every 7 days starting at t0.

	● With the default _multiplierDurations = [30d, 60d, 90d], ○ maxTierDuration = 90d. ○ After 30 weekly deposits the user has a full deposit stack and time = t0 + 203d. ● On the 31st weekly deposit at t0 + 210d, ○ _recordDeposit sees deposits.length == 30 >= MAX_DEPOSITS_PER_USER and calls _compactDeposits. ○ At that moment, deposit #X sits at timestamp t0 + (X-1) * 7d and has holding duration 210d - (X-1) * 7d, so deposits #1 through #18 all have duration >= 90d and Pass 1 merges them. ● The stack collapses to a single entry (1800 XVS, t0) plus deposits #19 through #30, ending at 13 entries, well below the cap, so Pass 2 never runs. ● Shortly afterward, at t0 + 215d, governance adds a 180d tier to incentivize even longer holds, updating _multiplierDurations to [30d, 60d, 90d, 180d] so maxTierDuration = 180d. ● The next getEffectiveStake call reads the merged entry with duration 215d, which is >= 180d, and applies the 180d tier multiplier to the full 1800 XVS. ○ In reality, only deposits whose individual duration at t0 + 215d is >= 180d should receive the 180d tier multiplier, which is deposits #1 through #6 with timestamps from t0 to t0 + 35d. ● The remaining 1200 XVS, from deposits #7 through #18 with individual durations between 95d and 173d, are overcredited with a 180d tier multiplier when they should still sit at the 90d tier multiplier.
Recommendation	Consider adjusting Pass 1 compaction under tier extension by tracking an amount weighted timestamp sum instead of the earliest timestamp, matching the logic already used in _compactByTier.
Status	Fixed, now replaces earliestTimestamp with an amount-weighted average timestamp in Pass 1 of _compactDeposits (and the same approach is used in _compactByTier) with the following caveats: 1. This situation is unlikely to occur currently as Venus has no plans to include a new multiplier duration. Additionally, a specific user can avoid a deposit compaction by keeping their pushed deposit stack below the MAX_DEPOSITS_PER_USER of 30 2. With this fix, there is an under-credit (taking this example) of deposits #1–6 (600 XVS), whereby individually, these deposits had ≥180d at t0+215d and legitimately deserved the 180d tier. With the fix they're credited at 90d due to the weighted average taken. This appears to be acceptable as a design choice to prevent over-estimation of scores for later pushed deposits.

3. Residual over-credit is still possible. For example, 1000 XVS at t=0 + 1 XVS at t=110d, merged at t=200d (both ≥90d). Weighted-avg timestamp ≈ 0.11d. If a 180d tier is added, the merged duration ≈ 199.89d ≥ 180d, all 1001 XVS gets the 180d multiplier, even though 1 XVS only qualifies for the 90d tier. This is an acceptable tradeoff per the code comment of "Minor score overestimate within a tier is acceptable since the score is only used for off-chain ranking."

Possible Further Considerations

1. **Snapshot the max tier into the Deposit struct**
   The Deposit struct already has an unused _reserved field. We can consider repurposing it as a per-deposit tier cap. At compaction time, cache every merged entry with maxTierCapSec = maxTierDuration (the current top-tier threshold). Then, in getEffectiveStake / _getMultiplier, clamp the holding duration to that snapshot before tier lookup:

2. **Minimize compaction so the lossy path runs less often**
   Currently, Pass 1 merges every max-tier deposit even when only one slot of the deposit stack is needed. We can consider only merging the oldest 2 deposits (or just enough to drop below MAX_DEPOSITS_PER_USER), preferring to leave individual entries intact.

[L01] _compactByTier weighted average merge causes possible score underestimation if setMultiplierTiers inserts a new threshold	
Contract	PrimeLeaderboard.sol
Severity	Low
Description	**Description** The PrimeLeaderboard._compactByTier fallback (Pass 2) collapses deposits within the same tier into a single entry whose timestamp is the amount weighted average of the constituents. This is in contrast to Pass 1, which merges only max tier deposits and explicitly preserves the earliest timestamp. When setMultiplierTiers later inserts a new threshold inside a previously existing tier's range, deposits that were previously merged under the old configuration are re-bucketed using only that weighted average, with no recovery of the original constituent timestamps. Consider a user with • Two deposits in old tier 2 (60d to 90d): 50 tokens at duration 89d and 150 tokens at duration 65d. Pass 2 collapses them into one entry at the weighted average duration (50 * 89 + 150 × 65) / 200 = 71d. • The admin then updates _multiplierDurations to [30d, 60d, 75d, 90d] with multipliers [1.3, 1.6, 1.8, 2.0], inserting a new tier 3 in the 75d to 90d band. Re-evaluating _getTierIndex against the merged entry places its 71d duration in the new tier 2 with multiplier 1.6.

- Had the constituents not been merged, the 89d deposit would now sit in new tier 3 at multiplier 1.8 while the 65d deposit would remain in new tier 2 at 1.6.

- The unmerged score at t = 0 would be 50 × 1.8 × 89 + 150 × 1.6 × 65 = 8010 + 15600 = 23610, while the merged score is 200 * 1.6 * 71 = 22720, a loss of 890 score units that arises purely from when the merge happened relative to the threshold insertion.

Pass 1 is unaffected because it keys off _multiplierDurations[length - 1] and preserves earliest timestamps. No state is corrupted, totalStaked still equals the sum of stored amounts, LIFO order is preserved.

JavaScript

```
  /**
     * @notice Set the multiplier tiers
     * @param durations Array of duration thresholds in seconds
     * @param multipliers Array of multiplier values (scaled by 1e18)
     * @custom:event Emits MultiplierTiersUpdated event
     * @custom:error Throw LengthMismatch if durations and multipliers
arrays have different lengths
     * @custom:error Throw InvalidValue if durations array is empty
     * @custom:error Throw InvalidMultiplierTiers if tiers are not in
ascending order or multipliers below base
     * @custom:access Controlled by ACM
     */
    function setMultiplierTiers(uint256[] calldata durations, uint256[]
calldata multipliers) external override {
        _checkAccessAllowed("setMultiplierTiers(uint256[],uint256[])");

        if (durations.length != multipliers.length) revert
LengthMismatch();
        if (durations.length == 0) revert InvalidValue();

        // Clear and set new tiers
        delete _multiplierDurations;
        delete _multiplierValues;

        // Validate and push in a single loop
        for (uint256 i; i < durations.length; ) {
            if (multipliers[i] < BASE_MULTIPLIER) revert
InvalidMultiplierTiers();
            if (i > 0) {
                if (durations[i] <= durations[i - 1]) revert
InvalidMultiplierTiers();
                if (multipliers[i] <= multipliers[i - 1]) revert
InvalidMultiplierTiers();
            }

            _multiplierDurations.push(durations[i]);
            _multiplierValues.push(multipliers[i]);
```

```
            unchecked {
                ++i;
            }
        }

        emit MultiplierTiersUpdated(durations, multipliers);
    }
```

Recommendation	Consider restricting setMultiplierTiers to changes that do not insert a new threshold inside/between an existing tier band. If not, provide sufficient time for users to perform deposit and withdrawal records within the XVSVault before a setMultiplierTiers change.
Status	Acknowledged, similar to point 1 and 2 of [M02], an insertion of a multiplier duration between currently existing durations is unlikely, with a slight underestimation being an acceptable trade off to make for a time weighted compaction of deposits

[L02] Incorrect __gap length in PrimeLeaderboardStorageV1	
Contract	PrimeLeaderboardStorage.sol
Severity	Low
Description	**Description** PrimeLeaderboardStorageV1 occupies 5 storage slots (_depositStacks, totalStaked, _multiplierDurations, _multiplierValues, and primeV2 packed together with stakersInitialized). Following Venus convention, the total slot count should be 50, requiring a __gap of 45. However, the current declaration uses 48, leaving 3 excess slots.

JavaScript

```
 // PrimeLeaderboardStorage — PrimeLeaderboardStorageV1
  mapping(address => IPrimeLeaderboard.Deposit[]) internal
_depositStacks; // slot 0
  mapping(address => uint256) public totalStaked;
// slot 1
  uint256[] internal _multiplierDurations;
// slot 2
  uint256[] internal _multiplierValues;
// slot 3
  address public primeV2;
// slot 4
  bool public stakersInitialized;
// packed into slot 4
```

```
  uint256[48] private __gap;
// should be 45
```

Impact
No functional impact at present, but the upgrade convention is broken. When new variables are appended in future upgrades, the baseline slot count will be miscalculated, potentially causing storage collisions.

Recommendation	Change the __gap length from 48 to 45 to align the total slot count with the established 50-slot convention.
Status	Fixed as recommended in commit ce7d47e,

[L03] removeMarket unnecessarily triggers a full score update round when no active members exist	
Contract	PrimeV2.sol
Severity	Low
Description	**Description** The PrimeV2.removeMarket() function requires sumOfMembersScore == 0 before a market can be removed. This precondition guarantees that no user currently holds a positive score in the market being removed, so the removal has no mathematical effect on any user's score in any other market. Reaching sumOfMembersScore == 0 in the first place requires either • A full burn of every member in that market, and those burns already perform the necessary per user accounting via _burnWithoutAccrual. • All users to have a zero score for that specific market i.e. full exit with no participants, in which case the score would have been updated via _updateScore

Description

JavaScript

```
 // PrimeV2 — removeMarket
  if (markets[market].sumOfMembersScore > 0) revert
MarketHasActiveMembers();
  // ... clear market state ...
  _queueScoreUpdates(); // sumOfMembersScore == 0, full round is
unnecessary
```

Impact
Despite this,removeMarket unconditionally invokes _queueScoreUpdates() at the end of execution, which sets pendingScoreUpdates = totalTokens and starts a fresh score update round that requires keepers to update every Prime holder. The work performed by that round is redundant because no score actually needs to be recomputed.

	A secondary less impactful consequence is that _queueScoreUpdates discards any score update round already in progress by emitting IncompleteRoundDiscarded and resetting the round counters. A market removal that is functionally a no op for scoring can therefore wipe out a legitimate in flight round started by an earlier parameter change such as updateAlpha or updateMultipliers, forcing keepers to restart that work from zero across all Prime holders.
Recommendation	Remove the _queueScoreUpdates() call from removeMarket, reserving it for parameter changes that genuinely affect user scores such as updateAlpha, updateMultipliers, and addMarket.
Status	Fixed in commit 027abf8, the _queueScoreUpdates invocation is removed from the removeMarket function.

[L04] accrueInterestAndUpdateScore(address) lacks access control	
Contract	PrimeV2.sol
Severity	Low
Description	**Description** The single-argument overload of accrueInterestAndUpdateScore is an external function with no access control, allowing anyone to call it and force a score recalculation for any user. Internally it calls accrueAllMarkets() (which triggers oracle price updates) and accrueInterestAndUpdateScoreWithoutAccrual(user) to recompute the user's score. JavaScript `// PrimeV2 — accrueInterestAndUpdateScore` `function accrueInterestAndUpdateScore(address user) external {` `_accrueInterestAndUpdateScore(user);` `}` **Impact** During periods of transient oracle price anomalies, an attacker can selectively call this function against a target user to force a score recalculation at an unfavorable price, suppressing that user's score and affecting their standing in Prime token distribution rankings.
Recommendation	Add access control to restrict calls to the primeLeaderboard contract only, or evaluate whether the function needs to remain public given that the two-argument overload already handles Comptroller hook calls.
Status	Fixed, although accrueInterestAndUpdateScore(user, market) can be called permissionlessly as well, similar to Prime (V1). It is arguably correct to keep scores updated during times of accurate oracle price returns, and so this is only an issue during large and inaccurate oracle fluctuations.

[I01] Permissionless Prime token claim allows enrolling unwilling users	
Contract	PrimeV2.sol

Severity	Informational
Description	**Description** The PrimeV2.claimPrime and claimPrimeBatch functions accept an arbitrary user address and mint a Prime token to that user without requiring msg.sender == user, a signature from the user, or any opt in mechanism. V2 also removed the V1 revocation flow that auto burned Prime when a user's stake fell below the threshold, so once minted the token persists until an ACM gated burn(user) call is made by an admin. This permissionless mint path creates several possible issues. • First, when tokenLimit is close to the count of eligible users, an attacker can preemptively claim Prime for low engagement eligible users (e.g. users just passing the effective mintThreshold), occupying scarce slots that those users may never actively use, while legitimate eligible users are crowded out and admins must manually burn to free slots. • Second, once isPrimeHolder[user] == true, every Comptroller policy hook in PolicyFacet.sol, across 10 call sites, runs _executeBoost and _updateScore for the user, and _calculateScore performs oracle write calls. This imposes additional gas cost on every mint, redeem, borrow, repay, seize, and transfer the user performs in any Prime market, with no way for the user to decline. • Third, a possible attacker can enumerate eligible addresses via getEffectiveStakeBatch, then calls claimPrimeBatch on the lowest ranked qualifying users first, exhausting tokenLimit before the admin's issue() calls land for the genuine top ranked cohort. The cap is consumed by off rank addresses, and the intended top users cannot receive Prime until burns free slots. Once a victim is enrolled, they begin accruing interest across active markets. That accrued interest remains claimable by the victim through claimInterest even after the Prime token is later burned and even after the underlying market is removed via removeMarket. The protocol therefore commits a payouts obligation to addresses that were enrolled without their consent and that were never part of the leaderboard's intended top N cohort, possibly leaking rewards

```javascript
  function claimPrime(address user) external nonReentrant whenNotPaused
{
        if (user == address(0)) revert InvalidAddress();
        if (pendingScoreUpdates > 0) revert ScoreUpdateInProgress();
        if (isPrimeHolder[user]) revert UserAlreadyHasPrimeToken();

        address leaderboard = primeLeaderboard;
        if (leaderboard == address(0)) revert LeaderboardNotSet();

        uint256 threshold = mintThreshold;
        if (threshold == 0) revert MintThresholdNotSet();

        uint256 deadline = mintDeadline;
        if (deadline != 0 && block.timestamp > deadline) revert
MintWindowClosed();
```

```
        uint256 score =
IPrimeLeaderboard(leaderboard).getEffectiveStake(user);
        if (score < threshold) revert EligibilityBelowThreshold(user,
score, threshold);

        _mint(user);
        _initializeMarkets(user);
    }


   function claimPrimeBatch(address[] calldata users) external
nonReentrant whenNotPaused {
        if (pendingScoreUpdates > 0) revert ScoreUpdateInProgress();

        address leaderboard = primeLeaderboard;
        if (leaderboard == address(0)) revert LeaderboardNotSet();

        uint256 threshold = mintThreshold;
        if (threshold == 0) revert MintThresholdNotSet();

        uint256 deadline = mintDeadline;
        if (deadline != 0 && block.timestamp > deadline) revert
MintWindowClosed();

        uint256 usersLength = users.length;
        _ensureMaxLoops(usersLength);

        _accrueAllMarkets();

        for (uint256 i; i < usersLength; ) {
            address user = users[i];

            if (!isPrimeHolder[user]) {
                uint256 score =
IPrimeLeaderboard(leaderboard).getEffectiveStake(user);
                if (score < threshold) {
                    emit SkippedIneligibleUser(user, score, threshold);
                } else {
                    _mint(user);
                    _initializeMarketsWithoutAccrual(user);
                }
            }

            unchecked {
                ++i;
            }
        }
    }
```

Recommendation	If permissionless minting must remain, consider on chain ranked set primitive such as a Merkle root of the top N posted by admin per epoch, if not, off-chain ranking systems must be robust enough to ensure up-to-date prime token holder rankings. Else, the mintThresholds and token limit must be large enough to allow sufficient prime token memberships and deter such an attack economically.
Status	Acknowledged, Venus has stated that this is a design choice as per the following reasoning: The function is permissionless by design, anyone who believes a target address qualifies can trigger the mint, but the target only receives a Prime token if getEffectiveStake(user) >= mintThreshold , where mintThreshold is governance-controlled. The leaderboard-driven distribution model assumes backend operations: each epoch the team computes the ranked cohort off-chain and uses the ACM-gated issue / issueBatch and burn / burnBatch paths to mint and revoke to kens for the correct set. tokenLimit and mintThreshold are tuned so that any opportunistic mint on a lower-ranked but threshold-eligible address remains with the operational headroom that admin burns can reclaim with the same epoch. No funds at risk, no enrollment of ineligible users, and the residual gas/interest implications only apply t o addresses that already meet the configured Prime eligibility bar.

[I02] Removal of auto burn on XVS deposit or withdrawal request within prime pool	
Contract	PrimeV2.sol
Severity	Informational
Description	**Description** In V1, the xvsUpdated callback re-evaluated _isEligible(totalStaked) on every XVS stake change and automatically burned revocable Prime tokens for users whose stake fell below the eligibility threshold. V2 removes this self cleaning behavior, as documented within the PrimeLeaderboard contract where it is stated that “Admin reads getEffectiveStakeBatch() off-chain, ranks users, and calls PrimeV2.issue()/burn() directly.” PrimeLeaderboard.xvsUpdated only records deposits and withdrawals and accrues interest, and PrimeV2 exposes no on chain check that re-evaluates whether an existing Prime holder still meets the threshold. Once isPrimeHolder[user] == true, the token can only be removed by an ACM gated burn(user) call. A user can thus stake enough XVS to raise their leaderboard effectiveStake to at least mintThreshold, The user or anyone else then calls claimPrime. Immediately afterward, the user requests withdrawal of all their XVS. Their boost score in every market drops to 0, so they earn nothing further, but their Prime slot remains

	occupied. The result is wasted capacity against tokenLimit and a persistent off chain support burden, since governance must periodically batch burn these holders, whereas V1 cleaned itself up through xvsUpdated.
Recommendation	Consider automatically burning a user's Prime token if they are no longer eligible during an XVS token deposit or withdrawal request. However, this would require a score update queue as it could change totalTokens being rejected while pendingScoreUpdates > 0. As such, likely no fix is required and a documentational note is sufficient.
Status	Acknowledged, Venus has stated that this is a design choice as per the following reasoning: The V2 distribution model is admin-managed by design: backend reads getEffectiveStakeBatch off-chain each epoch, ranks holders, and calls the ACM-gated issue/issueBatch and burn/burnBatch paths to keep the Prime cohort current. The V1 auto-burn path is intentionally removed because re-evaluating eligibility inside xvsUpdated would mutate totalTokens mid-round and conflict with the pendingScoreUpdates gate, as the audit itself notes. Vacated slots from withdrawals are reclaimed in the next epoch sweep; no funds are at risk and no ineligible user gains Prime in the interim.

[I03] setMintThreshold accepts a mintDeadline that is already in the past	
Contract	PrimeV2.sol
Severity	Informational
Description	**Description** The PrimeV2.setMintThreshold function does not validate that the supplied mintDeadline_ is strictly greater than block.timestamp. As a result, governance can accidentally pass an already elapsed timestamp, which immediately closes the mint window the moment the transaction executes. This introduces an operational mistake risk where a configuration call intended to open or adjust the mint window instead leaves it effectively shut, requiring a follow up governance action to recover. JavaScript `function setMintThreshold(uint256 mintThreshold_, uint256 mintDeadline_) external {` `_checkAccessAllowed("setMintThreshold(uint256,uint256)");` `emit MintThresholdUpdated(mintThreshold, mintThreshold_, mintDeadline_);` `mintThreshold = mintThreshold_;` `mintDeadline = mintDeadline_;` `}`

Recommendation	Consider adding a validation check in setMintThreshold that enforces mintDeadline_ > block.timestamp, reverting otherwise.
Status	Fixed in commit 7c05ebd, now reverts with the InvalidDeadline error if the mintDeadline_ input is non-zero and is less than or equal to the current block.timestamp

[I04] Unused ReentrancyGuardUpgradeable inheritance in PrimeLeaderboard	
Contract	PrimeLeaderboard.sol
Severity	Informational
Description	Description The PrimeLeaderboard contract imports, inherits, and initializes OpenZeppelin's ReentrancyGuardUpgradeable, yet the nonReentrant modifier is never applied to any function. The inheritance provides no protection while still occupying one permanent storage slot (the _status variable) in the upgradeable storage layout, adding bytecode size, and creating a misleading signal that reentrancy is already guarded. Additionally, xvsUpdated(address user) performs external calls into IPrimeV2.accrueInterestAndUpdateScore(user), which in turn calls IPrimeLiquidityProvider.accrueTokens(underlying) and oracle.updateAssetPrice / oracle.updatePrice. Any of those external contracts that introduces a callback or hook, now or in a future upgrade, could re enter PrimeLeaderboard.xvsUpdated. JavaScript `contract PrimeLeaderboard is` `IPrimeLeaderboard,` `AccessControlledV8,` `ReentrancyGuardUpgradeable,` `MaxLoopsLimitHelper,` `PrimeLeaderboardStorageV1` `{`
Recommendation	Consider applying the nonReentrant modifier to xvsUpdated(address user), which is the externally callable entrypoint that performs external calls.
Status	Fixed in commit b96facb, the nonReentrant modifier is added in the xvsUpdated function

[I05] initializeStakers does not validate timestamps in PrimeLeaderboard	
Contract	PrimeLeaderboard.sol
Severity	Informational
Description	**Description** The PrimeLeaderboard.initializeStakers function lacks timestamp validation, unlike Prime.setStakedAt which explicitly rejects future timestamps by reverting with InvalidTimestamp() when timestamps[i] > block.timestamp. • If a timestamp is set greater than block.timestamp, the getEffectiveStake calculation will underflow on the block.timestamp - uint256(d.timestamp) operation. As a result, the affected user is denied service from any scoring or claim flow until the seeded deposit ages past block.timestamp. • If a timestamp is set to 0 or any far past value, the resulting holdingDuration equals block.timestamp, which getEffectiveStake then caps at maxCapSeconds and multiplies by the highest multiplier. This produces the maximum possible effective stake for the seeded amount. JavaScript `function initializeStakers(` `address[] calldata users,` `uint256[] calldata amounts,` `uint64[] calldata timestamps` `) external {` `_checkAccessAllowed("initializeStakers(address[],uint256[],uint64[])");` `if (stakersInitialized) revert StakersAlreadyInitialized();` `if (users.length != amounts.length || users.length != timestamps.length) revert LengthMismatch();` `uint256 usersLength = users.length;` `_ensureMaxLoops(usersLength);`
Recommendation	Consider adding timestamp validation in initializeStakers consistent with the check applied in Prime.setStakedAt. Reject any entry where timestamps[i] > block.timestamp and additionally enforce a lower bound (such as rejecting zero timestamps or values older than a defined minimum) to prevent seeded deposits from receiving artificially inflated effective stake values.
Status	Fixed in commit 9ad4f33, where if the inputted timestamp is equal to zero or more than the current block.timestamp, the InvalidTimestamp error is triggered

[I06] Missing decimals validation in addMarket can permanently break score calculation	
Contract	PrimeV2.sol

Severity	Informational
Description	**Description** The PrimeV2.addMarket() function does not validate the underlying token's decimals when listing a new vToken. The _calculateScore logic performs a 10 ** (18 , decimals) exponentiation, which underflows whenever the underlying has decimals > 18. If governance ever lists such a market, every subsequent call to _calculateScore for that market will revert, making the market unusable within the Prime scoring system after it has already been added. JavaScript `function addMarket(address market, uint256 supplyMultiplier, uint256 borrowMultiplier) external {` `_checkAccessAllowed("addMarket(address,uint256,uint256)");` `if (markets[market].exists) revert MarketAlreadyExists();` `if (supplyMultiplier == 0 && borrowMultiplier == 0) revert InvalidMultipliers();` `bool isListed = InterfaceComptroller(corePoolComptroller).markets(market);` `if (!isListed) revert InvalidVToken();` `address underlying = _getUnderlying(market);` `if (vTokenForAsset[underlying] != address(0)) revert AssetAlreadyExists();` `markets[market] = Market({` `supplyMultiplier: supplyMultiplier,` `borrowMultiplier: borrowMultiplier,` `rewardIndex: 0,` `sumOfMembersScore: 0,` `exists: true` `});` `vTokenForAsset[underlying] = market;` `_allMarkets.push(market);` `_queueScoreUpdates();` `emit MarketAdded(market, supplyMultiplier, borrowMultiplier);` `}`
Recommendation	Considering Venus does not currently have any markets with underlying assets having decimals higher than 18, consider documenting such a behavior within code comments.
Status	Fixed in commit 9456254, now reverts if a market with an underlying token decimal greater than 18 is added with the UnsupportedUnderlyingDecimals error.

[I07] Removed APR view functions in PrimeV2	
Contract	PrimeV2.sol
Severity	Informational
Description	**Description** PrimeV2 removes three view functions that were exposed by the previous Prime contract and used by off-chain consumers: • calculateAPR: returned a populated APRInfo struct for a given user and market, including the user's current score, the market's total score, the XVS balance used for score, the capital figure, capped supply, capped borrow, supply cap in USD, borrow cap in USD, and the computed supplyAPR and borrowAPR. Frontends used it to display a user's live Prime APR for a market. • estimateAPR: returned the same APRInfo struct but for hypothetical inputs of borrow, supply, and staked XVS. Dashboards and yield calculators used it to project a user's APR if they changed their stake or position. • incomeDistributionYearly: returned the annualized income that would be distributed to Prime holders in a market, computed as blocksOrSecondsPerYear * getEffectiveDistributionSpeed(underlying).
Recommendation	Consider restoring equivalent on-chain helpers on PrimeV2, publish clearly documented replacement formulas using the getters that PrimeV2 does expose, or documenting that such view functions will no longer be supported
Status	Acknowledged, no fix implemented currently, Venus has stated that the off-chain computation helper view functions is TBD, depending on product requirements and specifications

[I08] Unused return values in _capitalForScore after removal of view helpers	
Contract	PrimeV2.sol
Severity	Informational
Description	**Description** The PrimeV2._capitalForScore() function returns cappedSupply and cappedBorrow in addition to the capital value. In the original Prime contract, these values were exposed through a Capital memory struct and consumed by view helpers that surfaced APR and capping information to off chain consumers. In PrimeV2, those view helpers have been removed, and the only remaining caller _calculateScore, uses just the capital field while discarding cappedSupply and cappedBorrow. As a result, these return values are computed but never read or accessible anywhere in the contract, leaving dead code that increases gas costs and reduces clarity.

```javascript
function _capitalForScore(
      uint256 xvsBalanceForScore,
      uint256 borrow,
      uint256 supply,
      address market
    ) internal view returns (uint256 capital, uint256 cappedSupply,
uint256 cappedBorrow) {
```

Recommendation	Consider • Refactoring _capitalForScore to return only the capital value, removing the cappedSupply and cappedBorrow return parameters along with the assignments that populate them. • If [I07] is to be fixed and future view functionality is expected to consume these values, document the intent explicitly or reintroduce a dedicated view helper rather than leaving unused return values in an internal function.
Status	Acknowledged, no fix implemented currently, Venus has stated that the off-chain computation helper view functions is TBD, depending on product requirements and specifications. Since the return values might be required for the helper functions, the fix is TBD.

[I09] Round gate on burn relies on keeper liveness to bound malicious withdrawal requests	
Contract	PrimeV2.sol
Severity	Informational
Description	**Description** When an admin parameter change queues a score update round, PrimeV2.burn and PrimeV2.burnBatch revert with ScoreUpdateInProgress until the round completes. During this window, a Prime holder can call XVSVault.requestWithdrawal, which immediately increments pendingWithdrawals and triggers xvsUpdated, causing both PrimeLeaderboard and PrimeV2 to treat the user's effective stake as already reduced. The user's tracked stake can drop below mintThreshold instantly, even though their XVS remains locked for the duration of the vault's lockPeriod. Admin cannot revoke the user's Prime status until the keeper finishes calling updateScores for every prime holder in the round. • In the typical case, the lockPeriod is long enough that the round completes first and the user gets burned before they can execute the withdrawal, so the attack reduces to temporary slot squatting under the tokenLimit.

	• However, in cases where the keeper is slow, the prime population is large relative to maxLoopsLimit, or the round becomes stuck due to an external revert in the updateScores call path, the round window can outlast the lockPeriod. In those cases, the user can call executeWithdrawal to recover their XVS while continuing to hold Prime status indefinitely. Note that this would not have been possible in Prime.sol because the V1 contract auto burned revocable holders inside xvsUpdated and allowed admin burns during a pending round. JavaScript `function burn(address user) external {` `_checkAccessAllowed("burn(address)");` `if (pendingScoreUpdates > 0) revert ScoreUpdateInProgress();` `if (!isPrimeHolder[user]) revert UserHasNoPrimeToken();` `_burn(user);` `}` `function burnBatch(address[] calldata users) external {` `_checkAccessAllowed("burnBatch(address[])");` `if (pendingScoreUpdates > 0) revert ScoreUpdateInProgress();`
Recommendation	Document the operational significance of keeper liveness explicitly in the protocol's operator runbook and in user facing documentation for PrimeV2. The documentation should make clear that the round gate on burn and burnBatch makes timely keeper execution a security requirement, not just a performance optimization. If not, all score queueing actions should always batch call the permissionless updateScores function.
Status	Acknowledged, Venus has stated that appropriate configuration and operational steps will be performed by the Venus admins, as per the following reasoning: Mitigation is operational: governance proposals that queue a round (updateAlpha, updateMultipliers, addMarket) are always followed by updateScores batches that drain pendingScoreUpdates back to zero. XVSVault lockPeriod (7 days) far exceeds typical round drain time (hours most given current totalTokens and maxLoopsLimit), so any user calling requestWithdrawal during the round is burned by admin before they can actually execute the withdrawal. Worst case (keeper outage or a stuck round that outlasts lockPeriod) is treated as an operational incident.

[I10] Redundant zero address check in xvsUpdated	
Contract	PrimeLeaderboard.sol
Severity	Informational

Description	**Description** The PrimeLeaderboard.xvsUpdated function rejects calls where user is the zero address, but the function is already restricted to be callable only by the XVSVault contract via the msg.sender == xvsVault gate. Because the vault drives this callback off of user actions that originate from msg.sender in the vault's own functions, and no real account is owned by the zero address, the vault cannot forward a zero address user to this hook under normal operation. JavaScript `function xvsUpdated(address user) external override {` `if (msg.sender != xvsVault) revert OnlyXVSVaultAllowed();` `if (user == address(0)) revert ZeroAddress();`
Recommendation	Consider removing the zero address validation from xvsUpdated, if not, it can be kept as a safeguard in case future upgrades allow xvsUpdated call for specific users outside of the msg.sender caller context.
Status	Acknowledged, Venus has stated that the check is kept as defense-in-depth against future XVSVault upgrades that may pass arbitrary user arguments to the callback.

[I11] Missing _ensureMaxLoops in addMarket allows _allMarkets to exceed the loop limit	
Contract	PrimeV2.sol
Severity	Informational
Description	**Description** In V1, addMarket calls _ensureMaxLoops(_allMarkets.length) after pushing the new market, so the addition is rejected if the resulting array size exceeds the configured loopsLimit. In PrimeV2.addMarket, this post push check is omitted, allowing the admin to push _allMarkets beyond loopsLimit without any revert at the time of addition. Once _allMarkets.length > loopsLimit, every operation that iterates the full markets array reverts inside the loop helper at the internal _ensureMaxLoops(marketsLength) call. This affects _burnWithoutAccrual, _initializeMarketsWithoutAccrual, _accrueAllMarkets, _accrueInterestAndUpdateScoreWithoutAccrual, getPendingRewards, and getPendingRewardsStatic. As a result, if the admin adds the (loopsLimit + 1)th market, the addition itself succeeds, but subsequent calls to claimPrime, claimPrimeBatch, issue, issueBatch, burn, burnBatch, updateScores, accrueInterestAndUpdateScore, and getPendingRewards all revert. The contract is effectively bricked until governance raises loopsLimit via setMaxLoopsLimit. While such a mitigation exists since the admin can raise the limit, the check in V1 is strictly safer than the fail silent then brick everything behavior in V2.

JavaScript

```
  function addMarket(address market, uint256 supplyMultiplier, uint256
borrowMultiplier) external {
        _checkAccessAllowed("addMarket(address,uint256,uint256)");

        if (markets[market].exists) revert MarketAlreadyExists();
        if (supplyMultiplier == 0 && borrowMultiplier == 0) revert
InvalidMultipliers();

        bool isListed =
InterfaceComptroller(corePoolComptroller).markets(market);
        if (!isListed) revert InvalidVToken();

        address underlying = _getUnderlying(market);
        if (vTokenForAsset[underlying] != address(0)) revert
AssetAlreadyExists();

        markets[market] = Market({
            supplyMultiplier: supplyMultiplier,
            borrowMultiplier: borrowMultiplier,
            rewardIndex: 0,
            sumOfMembersScore: 0,
            exists: true
        });

        vTokenForAsset[underlying] = market;
        _allMarkets.push(market);

        _queueScoreUpdates();

        emit MarketAdded(market, supplyMultiplier, borrowMultiplier);
    }
```

Recommendation	Consider restoring the post push _ensureMaxLoops(_allMarkets.length) check in addMarket
Status	Fixed in commit f73bf9e, now includes the _ensureMaxLoops post push check similar to Prime (V1)

[I12] setPrimeLeaderboard does not verify the new leaderboard is initialized	
Contract	PrimeV2.sol
Severity	Informational
Description	**Description** PrimeV2.setPrimeLeaderboard accepts any non zero address and immediately rewrites the primeLeaderboard pointer without checking the state of the target contract. The new leaderboard is only safe to use once its historical stakers have been seeded through PrimeLeaderboard.initializeStakers, which is the only path that populates

	depositStacks[user] and totalStaked[user]. After finalizeInitialization() is called, seeding is locked permanently. If the setter is pointed at a fresh, unseeded leaderboard, getEffectiveStake(user) returns 0 for every user because every deposit stack is empty. Every call to claimPrime then reverts with EligibilityBelowThreshold(user, 0, mintThreshold), soft locking the entire mint window until the admin manually batches initializeStakers and finalizes. JavaScript `/** * @notice Set the PrimeLeaderboard contract address used for permissionless mint eligibility * @param primeLeaderboard Address of PrimeLeaderboard contract * @custom:event Emits PrimeLeaderboardSet event * @custom:error Throw InvalidAddress if address is zero * @custom:access Controlled by ACM */ function setPrimeLeaderboard(address primeLeaderboard_) external { _checkAccessAllowed("setPrimeLeaderboard(address)"); if (primeLeaderboard_ == address(0)) revert InvalidAddress(); emit PrimeLeaderboardSet(primeLeaderboard, primeLeaderboard_); primeLeaderboard = primeLeaderboard_; }`
Recommendation	Consider including a check within setPrimeLeaderboard that requires the new leaderboard to already havestakersInitialized == true before allowing updates
Status	Acknowledged, no fix implemented. Venus has stated that appropriate configuration and operational steps will be performed by the Venus admins, as per the following reasoning: setPrimeLeaderboard is ACM-gated and only callable by governance. The PrimeLeaderboard migration flow (initializeStakers → finalizeInitialization → vault wiring) is already documented at the leaderboard, and setPrimeLeaderboard on PrimeV2 is the last step in that operational order. Any deviation would surface immediately when the first claimPrime reverts with EligibilityBelowThreshold(user, 0, threshold), before any user state is mutated (the revert occurs before _mint).

[I13] Missing zero address validation in issue and issueBatch	
Contract	PrimeV2.sol
Severity	Informational
Description	**Description**

	claimPrime validates the user parameter with if (user == address(0)) revert InvalidAddress(), but the admin functions issue and issueBatch both omit this check. Passing address(0) would set isPrimeHolder[address(0)] = true, consuming one tokenLimit slot. Note: This behavior is inherited and present in Prime (V1) as well JavaScript `// PrimeV2 — claimPrime` `if (user == address(0)) revert InvalidAddress(); // validated` `// PrimeV2 — issue` `function issue(address user) external {` `_checkAccessAllowed("issue(address)");` `if (pendingScoreUpdates > 0) revert ScoreUpdateInProgress();` `if (isPrimeHolder[user]) revert UserAlreadyHasPrimeToken();` `// no zero address check` `_mint(user);` `_initializeMarkets(user);` `}` **Impact** Passing a zero address wastes one tokenLimit slot and requires an admin burn(address(0)) call to recover. This is an operational mistake risk rather than a security vulnerability.
Recommendation	Add a zero address check to both issue and issueBatch to match the validation already present in claimPrime.
Status	Fixed in commit 4449a25, now reverts with the InvalidAddress if the inputted user is the zero address for both issue/issueBatch

[I14] Missing admin setters for oracle, primeLiquidityProvider, and corePoolComptroller	
Contract	[ContractName.sol]
Severity	Informational
Description	**Description** The three critical addresses oracle, primeLiquidityProvider, and corePoolComptroller are set only once in initialize with no admin functions provided to update them. The previous Prime.sol contract exposed corresponding setters such as setPrimeLiquidityProvider and setOracle.

	JavaScript ``` // PrimeV2 — initialize primeLiquidityProvider = primeLiquidityProvider_; corePoolComptroller = corePoolComptroller_; oracle = ResilientOracleInterface(oracle_); // no corresponding setters ``` Note: This behavior is inherited and present in Prime (V1) as well **Impact** If the oracle, PLP, or comptroller needs to migrate to a new address, a proxy upgrade is required, increasing operational complexity and upgrade risk.
Recommendation	Consider adding ACM-gated setter functions for these three addresses, following the pattern established in the previous Prime.sol, or explicitly document that address changes require a proxy upgrade.
Status	Acknowledged, Venus has stated that this is a design decision, similar to Prime (V1) as per the following reason: V1 has identical behavior — same three addresses are set once in initialize with no setters, and it has run in production for years without ever needing migration. All three underlying contracts (ResilientOracle, PrimeLiquidityProvider, comptroller) are themselves proxy-upgradeable, so implementation changes don't require updating PrimeV2's stored addresses. Any rare architectural migration that does require swapping a proxy address can be handled via PrimeV2's own proxy upgrade.